3Dレールシューティング

アズレイド

プログラム説明資料

小泉 羚

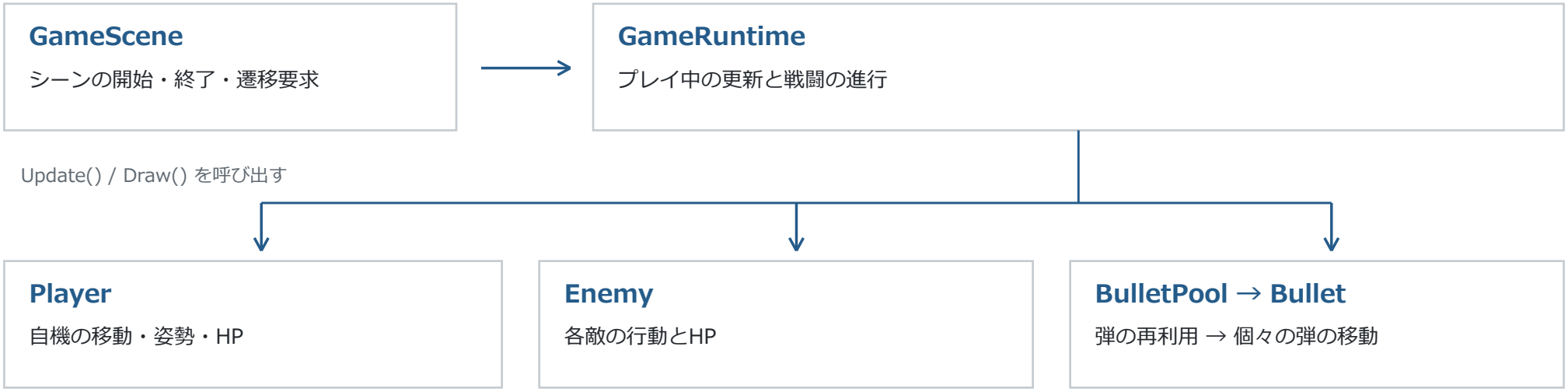
LE3B_08_コイズミ_リョウ

目次

内容	ページ
1. ゲーム全体の処理	3
2. 出撃中の移動とカメラ	4
3. 機体の分解と落下	5
4. 炎の配置とアニメーション	6
5. 照準から発射方向を求める	7
6. チャージ弾の範囲攻撃	8
7. 弾の再利用	9
8. クラスごとの役割	10
まとめ	11

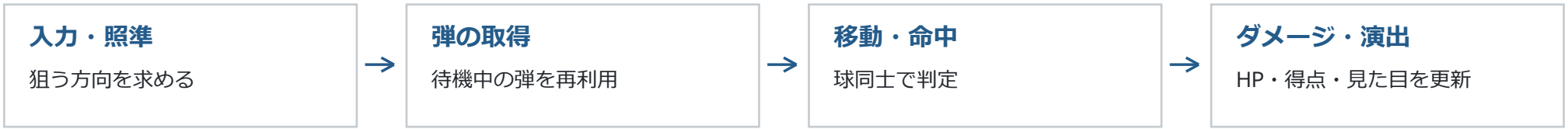
1. ゲーム全体の処理

自動で前進しながら照準を動かし、敵編隊とボスを撃破する3Dレールシューティングです。
GameSceneが更新と描画を呼び出し、GameRuntimeが自機・敵・弾・カメラを管理します。



主なクラスを抜粋しています。実線は、更新の呼び出しやオブジェクトの保持を表しています。

射撃したときの流れ

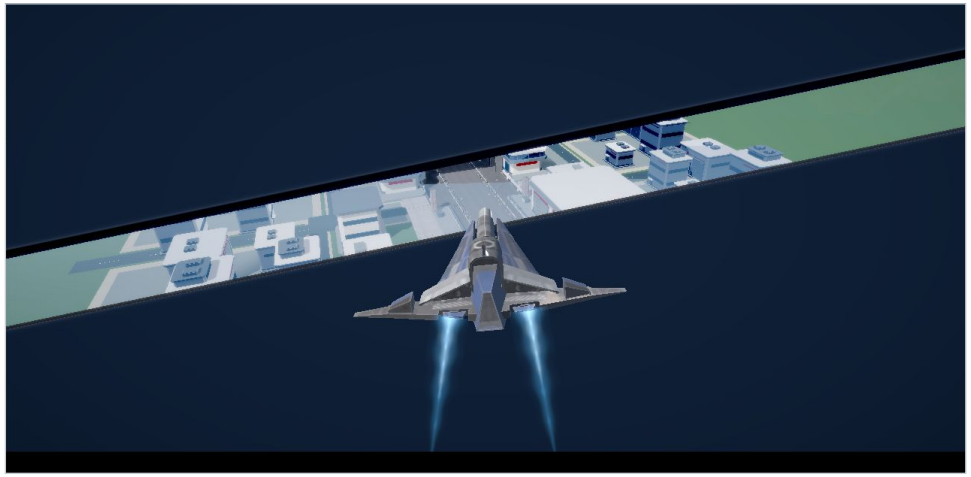


命中判定はGameRuntimeCollision.cpp、弾の取得・返却はBulletPool.cppで行います。

参照：GameScene.cpp / GameRuntime.h / GameRuntimeCollision.cpp

2. 出撃中の移動とカメラ

ステージ選択からゲーム開始へ移る際、背景や機体の位置が急に変わるのが不自然でした。
同じステージを描き続け、機体が隙間を抜けた後に通常の飛行へつなげています。



隙間へ向かう機体を、後ろから追う



通過後も同じ機体と背景のまま降下する

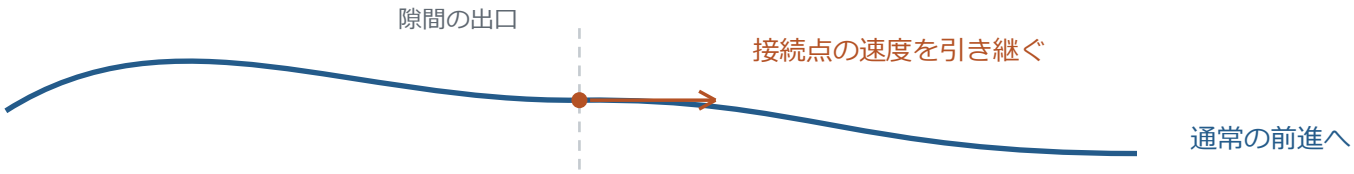
通過前：位置と速度を決めて補間

エルミート補間で通過点をつなぎます。
各点の速度も指定し、急な方向転換を抑えています。

通過後：出口の速度を引き継ぐ

出口の位置と速度を引き継ぎ、5次曲線で通常の飛行へつなぎます。
カメラも進行方向に合わせ、追走距離を徐々に戻します。

飛行軌道の模式図



参照：GameRuntimeEntry.cpp / StageFlightSpline・StageFlightApproach・UpdateStageOverview

3. 機体の分解と落下

撃墜時に機体が一瞬で消えると、壊れた様子が伝わりにくいため、部品が残るようにしました。
失速 → 分解 → 接地の順に進め、残骸が見える状態でリザルトを表示します。



1　失速して傾く



2　胴体・機首・左右の翼へ分解



3　路面に部品が残る

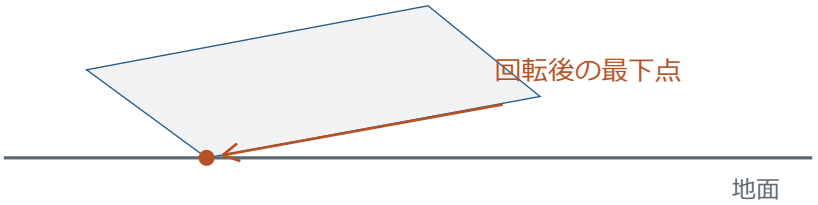
モデルを4つの部品に分ける

元モデルの頂点・UV・材質を使い、三角形の位置に応じて胴体・機首・左右の翼に分けます。
部品のモデルはゲーム開始時に用意し、撃墜時の生成を避けています。

初速・回転速度・反発の強さをランダムに変え、
毎回同じ壊れ方にならないようにしています。

回転後の形状で接地位置を求める

部品の頂点を回転させ、最も低い点が地面にめり込まないように位置を補正します。



参照：GameRuntimeDefeat.cpp / InitializePlayerDefeatDebris・DebrisBottom・UpdatePlayerDefeat

4. 炎の配置とアニメーション

炎が部品とは別の場所に浮いて見えたため、火元の位置と画像の動かし方を見直しました。
炎の画像には配布素材を使用し、部品への配置とアニメーションの処理を実装しています。



部品の回転に合わせて火元を移す

部品の三角形から、表側・裏側の火元の候補を選びます。
回転後に上側になる点を使います。



●：火元の候補（模式図）

画像の下端を固定し、上部だけを揺らす

上部ほどUVを大きくずらし、火元になる下端は動かさないようにしています。



隣り合うコマを混ぜ、切り替わりを滑らかにする

Object3d.PS.hlsl / DrawFlame

```
187  float3 emission = lerp(SampleFlameFrame(uv, current),
188      SampleFlameFrame(uv, fmod(current + 1.0f, 80.0f)), frac(frame));
```

lerpで、現在のコマと次のコマの色を混ぜます。

配置：GetDefeatFireOrigin（GameRuntimeDefeat.cpp）／ 炎素材：同梱のwreck_fire_LICENSE.txt参照

5. 照準から発射方向を求める

照準は2Dの画面座標、自機と敵は3D空間の座標で管理しています。
カメラと同じ向きに弾を飛ばすだけでは、銃口の位置によって照準からずれます。

画面の座標から3D空間の方向を求める

1 **照準の画面座標を変換**

画面座標を -1〜1 の範囲に変換する

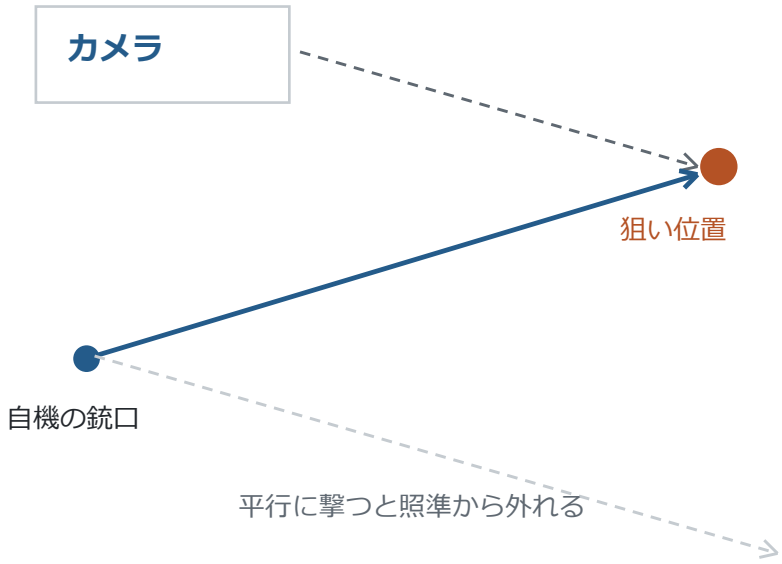
2 **逆射影で手前・奥の2点を求める**

ビュー射影行列の逆行列を使う

3 **2点を結び、照準が指す方向を求める**

敵の奥行きから、3Dの狙い位置を決める

上から見た位置関係（模式図）



銃口から狙い位置へのベクトルを正規化する

GameRuntime.cpp / CalculateAimDirection

```
6649  return Math::Normalize({ aimPoint.x - origin.x, aimPoint.y - origin.y, aimPoint.z - origin.z });
```

aimPoint：3Dの狙い位置　origin：銃口の位置　／　ロック中の敵がいない場合は、一定の奥行きで狙い位置を決める

6. チャージ弾の範囲攻撃

通常射撃だけでは単調だったため、チャージ弾に爆風を付けました。
狙った敵の周囲も巻き込めるよう、直撃と爆風の判定を分けています。



命中前：中央の敵を狙う

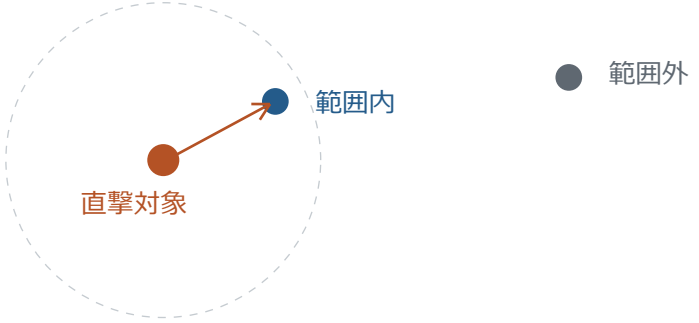


命中後：小型機3機を巻き込んで撃破

爆風の中心からの距離で選ぶ

爆風の半径内にいる敵だけにダメージを与えます。
画面上で重なっていても、3D空間で離れている敵には届きません。

直撃した敵は爆風の対象から外し、ダメージの重複を防ぎます。
爆風による連鎖爆発も発生させません。



判定の模式図。実際は3D空間の距離を使います。

比較用に敵配置を固定。判定：GameRuntimeCollision.cpp / CheckBulletEnemyCollisions・ApplyChargeSplash

7. 弾の再利用

連射中に生成・破棄を繰り返さないよう、オブジェクトプールで弾を再利用しています。
弾と管理用リストのノードをゲーム開始時に用意し、発射時に待機中から使用中へ移します。



BulletPool.cpp / Acquire

```
29  const auto entry = std::prev(available_.end());
30  active_.splice(active_.end(), available_, entry);
```

spliceでノードそのものを移し、弾のアドレスを変えずに管理先を切り替えます。

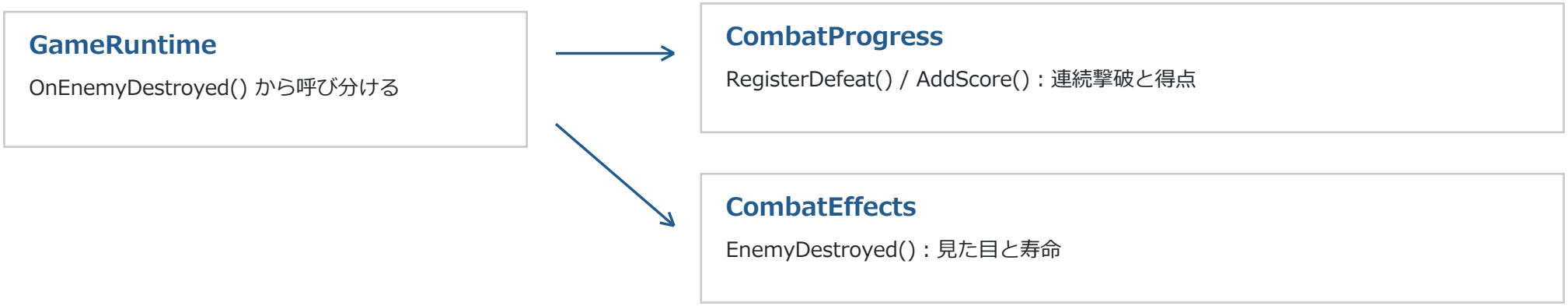
返す前に、関連する情報も解除する

追尾対象や回避済み判定を解除してから返却し、次の弾に前回の情報を残さないようにしています。
空きがない場合は新規確保せず、取得失敗を記録します。弾の初期化は取得側で行います。

参照：BulletPool.cpp / GameRuntime.cpp　所有にはunique_ptrを使い、利用側にはポインタを返す

8. クラスごとの役割

GameRuntimeに集中していた状態と処理を、役割ごとのクラスに分けています。
敵を撃破したときは、得点の更新と撃破エフェクトの呼び出しを別々に行います。



得点はAddScore()、演出はAddEnemyHitEffect()を経由します。

主なクラスと管理する情報

CombatProgress	スコア・連続撃破・倍率
FeverState	ゲージ・発動中の残り時間
BulletPool	使用中・待機中の弾
RailScenery	背景の配置と可視判定

GameRuntimeに残っている処理

HUD・ポーズ・出撃演出の状態は、まだGameRuntimeに残っています。
画面表示と入力が混ざる部分を引き続き整理します。

整理の方針

描画や演出のクラスから、得点やHPを直接変更しないようにしています。

関係図は一部を抜粋。各クラスはGameRuntimeのメンバとして保持しています。

まとめ

出撃・破壊の演出

機体の位置と速度を出撃後の飛行に引き継ぎ、カメラも通常の追走へ戻しています。
撃墜後は部品を残し、回転後の形状に合わせて接地位置と火元を求めています。

射撃・弾の管理

照準から3D空間の発射方向を求め、チャージ弾には範囲攻撃を追加しました。
弾をプールから取り出して再利用し、連射中の生成・破棄を避けています。

クラスの整理

得点・フィーバー・弾・背景の管理を、役割ごとのクラスに分けています。
演出とゲームのルールを分け、変更する箇所を見つけやすくしています。

次に進めること

敵配置とスキルのバランス調整、新規ステージの制作を進めます。
あわせて、GameRuntimeに残るHUD・ポーズ・出撃演出の処理を整理します。